

tmux documentation

tmx Shell-Session Resume — End-User Master Manual

Revision: 1 **Last modified:** 2026-05-22T14:30:00Z **Authority:** vasic-digital tmux project
Maintainer: milosvasic **Scope:** End-user (operator) master manual for the v1.0.9 shell-session-resume / SSH-dispatch / Go state-daemon feature triple. Worked examples copy-paste-runnable on macOS + Linux.

Table of contents

1. [Introduction — what changed in v1.0.9](#)
 2. [Quick start — fastest path from clone to first resume](#)
 3. [Daily operations](#)
 - 3.1 [Scenario A — your morning](#)
 - 3.2 [Scenario B — deep-dive on a topic over multiple days](#)
 - 3.3 [Scenario C — SSH straight into a remote session](#)
 - 3.4 [Scenario D — skip the prompt for one shell](#)
 - 3.5 [Scenario E — inspect / reset state by hand](#)
 4. [Reference](#)
 5. [Anti-bluff covenant \(§11.4\)](#)
 6. [Troubleshooting matrix](#)
 7. [Upgrading from pre-v1.0.9](#)
 8. [Last verified](#)
-

1. Introduction — what changed in v1.0.9

v1.0.9 ships **four new things** that together let you "pick up where you left off" without typing wrapper commands by hand:

1. `scripts/tmx-shell-init.sh` — the rc-side prompt is now one project-owned file you `.` (source) from `.bashrc/.zshrc`. No more drifting hand-pasted snippets. It's safe on SCP / rsync / IDE shells: the script silently exits when stdin is not a TTY.
2. `scripts/tmx-state-bin` (Go) — a tiny daemon that stores per-session last-cwd in `~/.tmx/state.json` with atomic writes and `fcntl(F_SETLKW)` locking. Five subcommands: `record` / `recall` / `list` / `forget` / `version`.
3. **Wrapper integration** — `tmx new -s NAME` now consults the state daemon and starts the pane in the recorded cwd. A tmux hook records the cwd back on detach / session-closed.
4. **SSH dispatch** — install once on each remote with `bash scripts/tmx-ssh-install.sh <user>@<host>`; afterwards `ssh <host>-tmx <session>` lands you straight inside that session with the right cwd. `ssh <host>-tmx` (no session arg) opens an ordinary login shell.

Why this matters in one sentence: **you stop typing** `tmx attach -t work` or **guessing what directory you were in yesterday** — the wrapper + daemon + ssh dispatcher do it all for you, transparently.

2. Quick start — fastest path from clone to first resume

This walk-through assumes a fresh checkout on macOS (the steps for Linux are identical except `sudo bash scripts/install_deps.sh` is required before `setup.sh`).

1. Clone the repo.

```
git clone --recurse-submodules git@github.com:vasic-digital/
tmux.git ~/Projects/tmux
cd ~/Projects/tmux
```

2. Build + verify + install. This also generates `scripts/tmx-shell-init.sh`

(from the template) AND `scripts/tmx-state-bin` (from Go), and appends

the source line to `~/.bashrc` and `~/.zshrc`.

```
bash scripts/setup.sh
```

Expect: SUMMARY: PASS=24 FAIL=0 SKIP=2 → GREEN.

3. Open a fresh shell.

```
exec $SHELL -l
```

You will see the new prompt:

[tmx] Enter session name (blank or "default" = bare shell):

4. Type a session name (e.g. 'work') and press Enter.

```
work
```

You are now inside tmx session 'work', pane started at \$HOME because

this is the first time you've used this name.

5. Move around.

```
cd /tmp
```

```
pwd      # /tmp
```

6. Detach (return to your outer shell).

Press Ctrl-b d. You'll see "[detached (from session work)]".

7. Restart a shell — the prompt appears again.

```
exec $SHELL -l
```

[tmx] Enter session name (blank or "default" = bare shell): work
pwd

```
# /tmp                                ← cwd restored from ~/.tmx/
state.json !
```

That's it. The "restored cwd" is the whole point. The state file lives at `~/ .tmx/state.json` and is updated automatically by a tmux hook each time you detach.

3. Daily operations

3.1 Scenario A — your morning

You log in for the day. You want to resume yesterday's work.

```
# Fresh terminal opens. .zshrc fires tmx-shell-init.sh.
[tmx] Enter session name (blank or "default" = bare shell): work
# → ATTACHED to session 'work' (it survived overnight).
# → If it didn't survive (host rebooted), tmx new -s work -c
    <recalled-pwd>
#   creates it fresh at the right cwd.
pwd
# /Users/milosvasic/Projects/tmux/docs      ← where you were
    last night.
```

If the prompt feels noisy, see scenario D for the per-shell opt-out.

3.2 Scenario B — deep-dive on a topic over multiple days

You're investigating a tricky bug. You'll come back to this many times.

```
# Day 1, morning.
$ exec $SHELL -l
[tmx] Enter session name ...: investigate
# inside the new pane:
$ cd /Users/milosvasic/Projects/tmux/scripts
$ less tmx-state/main.go
# read for a while; take notes elsewhere.
$ # Ctrl-b d    (detach)

# Day 1, afternoon.
$ exec $SHELL -l
[tmx] Enter session name ...: investigate
# attached, pane is back where you left it.
$ pwd
/Users/milosvasic/Projects/tmux/scripts
$ cd tmx-state      # narrow further
$ less main.go
$ # Ctrl-b d

# Day 2.
$ exec $SHELL -l
[tmx] Enter session name ...: investigate
```

```
$ pwd
/Users/milosvasic/Projects/tmux/scripts/tmx-
state ← still right where you stopped.
```

The state file records the deepest pane's cwd at detach time. If you had two panes open with different clds, the LAST detach wins (the tmux hook fires per-pane on close as well).

3.3 Scenario C — SSH straight into a remote session

Setting it up once:

```
# On your mac (mistborn.local), with a working `ssh
  milosvasic@nezha.local`:
$ cd ~/Projects/tmux
$ bash scripts/tmx-ssh-install.sh milosvasic@nezha.local
# 8 steps, idempotent. At the end:
#   local alias:      Host nezha.local-tmx
#   remote dispatch /home/milosvasic/Projects/tmux/scripts/tmx-
  ssh-dispatch.sh
#   verification PASS
```

Using it day-to-day:

```
# Empty command — login shell:
$ ssh nezha.local-tmx
[tmx] Enter session name ....: work
# → inside tmx session 'work' on nezha, cwd restored.

# Or skip the prompt by passing the session name directly:
$ ssh nezha.local-tmx work
# → straight inside the session, no prompt.
# → first time? created at the cwd `tmx-state recall work`
  returned;
# → subsequent? attached to the existing session.
```

The dispatch key has no-port-forwarding, no-X11-forwarding, no-agent-forwarding in authorized_keys, so even if the key file leaked the worst an attacker could do is open tmx sessions — no shell escape, no port forwarding.

3.4 Scenario D — skip the prompt for one shell

Sometimes you just want a bare shell with no tmx wrapping.

```
# Option 1: type 'default' (literal token) when prompted.
[tmx] Enter session name (blank or "default" = bare shell):
  default
$ # bare shell, no tmx, no nesting.

# Option 2: press Enter (blank input).
[tmx] Enter session name (blank or "default" = bare shell):
```

```

$ # same — bare shell.

# Option 3: per-shell opt-out via env var.
$ TMX_SKIP=1 bash -l
# .bashrc runs, but tmx-shell-init.sh exits on the TMX_SKIP
  guard.
$ # bare shell.
$ unset TMX_SKIP

# Option 4: permanently set TMX_SKIP=1 in your rc BEFORE the
  source line.
#   (e.g. on a build server where you never want tmx).

```

3.5 Scenario E — inspect / reset state by hand

```

# What sessions does my state file know about?
$ tmx-state list
build          /Users/milosvasic/Projects/tmux/scripts
               1748189000
investigate    /tmp
               1748190100
work          /Users/milosvasic/Projects/tmux/docs
               1748190500

# Pretty-print the raw file.
$ cat ~/.tmx/state.json | python3 -m json.tool

# Forget one session (idempotent — no error if absent).
$ tmx-state forget investigate
$ tmx-state recall investigate
$ echo $?
1

# Nuclear option: wipe everything.
$ rm ~/.tmx/state.json
# The next tmx new -s X rebuilds the directory/file
  automatically.

```

4. Reference

The full operator-grade documentation lives in:

Topic	Path
Shell integration (rc snippet)	docs/guides/tmx-shell-integration.md
Go state daemon CLI	docs/guides/tmx-state.md
SSH dispatch	docs/guides/tmx-ssh-dispatch.md

Topic	Path
Original install / OS notes	docs/guide/README.md
Design spec	docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md
§11.4.18 script companions	docs/scripts/tmx-shell-init.md, tmx-state.md, tmx-ssh-install.md, tmx-ssh-dispatch.md

Wrapper command reference unchanged:

```
tmx new -s NAME      # create (or attach if exists), restored cwd
tmx attach -t NAME   # attach (fails if session does not exist)
tmx ls               # list your sessions
tmx kill             # alias for kill-session
tmx kill-server      # nuke all your sessions
```

Per-session resource overrides (unchanged from v1.0.x):

```
TMX_MEM=8G      tmx new -s heavy      # Linux: 8 GB MemoryMax
TMX_CPU=400     tmx new -s build       # Linux: 400% CPUQuota
TMX_CPU_HARD_SEC=3600 tmx new -s timeboxed # Darwin: 1 hour
                RLIMIT_CPU
```

5. Anti-bluff covenant (§11.4)

This project ships under a hard rule: **every test that says PASS must have captured positive runtime evidence that the feature actually works for an end user.** No green-because-no-crash. No green-because-config-exists.

For v1.0.9 the new tests are 27–40 plus 41 (the doc-render challenge):

Test What it asserts

- 27 tmx-state record → recall round-trip + real tmux hook firing
- 28 typing default at the prompt does NOT spawn a tmux server
- 29 blank input also does NOT spawn a tmux server
- 30 non-TTY stdin (</dev/null) returns 0 in < 5 s, no prompt
- 31 local SSH dispatcher creates the session
- 32 real SSH against nezha.local creates the session + restores cwd
- 33 10 parallel tmx-state record calls all land — JSON still parses
- 34 re-running the SSH installer is a no-op (no duplicate AK lines)
- 35 session names with ;, /, . . are rejected
- 36 dispatcher with SSH_ORIGINAL_COMMAND="echo hi" exits 1, no tmux
- 37 nested invocation under \$TMUX skips silently
- 38 recall returns a deleted path → wrapper falls back to \$HOME
- 39 chmod 000 ~/.tmx → tmx new still works
- 40 case "\$(uname -s)" branches PASS on both Linux and Darwin
- 41 every guide + manual renders to HTML + PDF (this very document!)

Paired §1.1 mutations (in `scripts/tests/meta_test_false_positive_proof.sh`) prove the gates themselves are not bluffs: e.g. M20 strips the `-t 0` guard from `tmx-shell-init.sh` and asserts test 30 FAILs.

If you ever suspect a bluff (e.g. a feature says PASS in tests but doesn't work for you), the first thing to look at is the `[evidence]` lines in the test output:

```
$ bash scripts/tests/27_state_persistence.sh
[evidence] iter=1 recall1=/tmp/tmx-test-18-target-12345
          pane_path=/tmp/tmx-test-18-target-12345 recall2=/tmp/tmx-
          test-18-hook-12345-1
[evidence] iter=2 recall1=... pane_path=...
[evidence] iter=3 recall1=... pane_path=...
[evidence] HOST_OS=Darwin reliability_hash=8c4a...
PASS 18 tmx-state cwd persistence end-to-end (3/3 iterations
          identical, hook updated state)
```

Each `[evidence]` line is the raw captured fact. If a PR removes the evidence captures, code review must reject it under §11.4.2.

6. Troubleshooting matrix

Symptom	Likely cause	First thing to try
Prompt missing on login	rc not sourced	<code>exec \$SHELL -l</code> ; if still missing, re-run <code>bash scripts/setup.sh</code>
Prompt appears in SCP / rsync (BLOCKING)	Pre-v1.0.9 hand-pasted snippet still in your rc	Remove the old block; ensure ONLY the <code>scripts/tmx-shell-init.sh</code> line remains
Cwd not restored on <code>tmx new -s work</code>	<code>tmx-state-bin</code> not built or not on PATH	<code>bash scripts/setup.sh</code> ; which <code>tmx-state</code> should print a path
Cwd restored to a path that no longer exists	Stale state — directory was moved	<code>tmx-state forget work</code> then re-create the session
<code>ssh nezha.local-tmx work</code> hangs	Network or sshd issue	<code>ssh -vv nezha.local-tmx work 2>&1 head -50</code> ; <code>journalctl -u sshd</code> on remote
<code>ssh nezha.local-tmx work</code> rejects a valid name	Dispatcher regex mismatch	Names: <code>[A-Za-z0-9_-]{1,64}</code> . Avoid spaces, slashes, semicolons
Two Host <code>nezha.local-tmx</code> blocks in <code>~/.ssh/config</code>	Manual edit then installer added another	Hand-edit; keep one. Installer is idempotent only when the block is intact
<code>[tmx] invalid session name 'work;ls'</code>	Name contains a metachar	Fix the name — by design we reject shell-metachars (security)
Pre-v1.0.9 SSH config still references absolute paths	Old snippet had <code>command=/abs/path/...</code>	<code>bash scripts/tmx-ssh-install.sh --uninstall <target></code> then re-install
The state file looks corrupt	Disk full or external process clobbered it	<code>mv ~/.tmx/state.json ~/.tmx/state.json.bad</code> ; <code>tmx-state</code> rebuilds an empty one

7. Upgrading from pre-v1.0.9

If you're upgrading from v1.0.8 or earlier:

1. Pull the new code:

```
cd ~/Projects/tmux
git pull --ff-only
git submodule update --init --recursive
```

2. Re-run setup (this is idempotent — it generates the new files, builds the Go binary, and updates your rc snippet):

```
bash scripts/setup.sh
```

3. **Important:** remove any LEGACY hand-pasted snippet from your `.bashrc` / `.zshrc` that calls `read -r` directly. The legacy block typically looks like:

```
# OLD — REMOVE
if [ -t 0 ]; then
    printf 'Enter session: '
    read -r SESSION
    [ -n "$SESSION" ] && tmx new -s "$SESSION"
fi
```

The new block sources the project-owned file instead:

```
# NEW — KEEP
[ -r "$VDIGITAL_TMUX_DIR/tmx-shell-init.sh" ] && .
"$VDIGITAL_TMUX_DIR/tmx-shell-init.sh"
```

4. Open a fresh shell. The new prompt is functionally similar; the behaviour differences are:
 - `default` (literal) now means `SKIP` (used to mean "create default session");
 - blank input also means `SKIP` (used to be the default-session shortcut);
 - `SCP` / `rsync` / `IDE` shells no longer hang.

If you previously had a wrapper that ran `tmx new -s default` automatically, you now need to type `default-session` (or any other non-`default` name) to get the same behaviour.

7a. Uninstall (v1.0.11+)

The dedicated uninstall entry point removes EVERYTHING `setup.sh` installed and leaves operator data untouched:

```
cd ~/Projects/tmux
bash scripts/uninstall.sh
```

This calls `setup.sh --uninstall` under the hood (single source of truth), which:

1. Strips the — `vasic-digital` optimized `tmux` — fenced block from `~/.bashrc` and `~/.zshrc`.

2. Strips any LEGACY hand-pasted `if command -v tmx ...; then ...; fi` block (operators upgrading from pre-v1.0.9 had this risk).
3. Removes `~/ .tmux.conf` ONLY if it carries the `vasic-digital` optimized `tmux` configuration marker; pre-existing operator-owned `~/ .tmux.conf` is preserved (a backup was created at install time as `~/ .tmux.conf.pre-vasic-digital`).
4. Removes the generated files in the project: `scripts/tmx`, `scripts/tmx-shell-init.sh`, `scripts/tmx-state-bin`. All three are produced from tracked templates / source at install time, so removing them is safe — the next `setup.sh` regenerates them.

Also purge per-session last-pwd state

By default, `~/ .tmx/` (the cwd-memory state file) is preserved per §9 zero-risk-data-safety. To also remove it:

```
bash scripts/uninstall.sh --purge-state
```

Clean-slate reinstall (automatic, v1.0.11+)

Every `bash scripts/setup.sh` invocation now runs the `uninstall` logic SILENTLY as step 0 before re-installing. You no longer need to manually uninstall before upgrading — just run `setup.sh` and the stale generated artefacts (old wrapper, missing `init.sh`, half-installed `rc` block) are cleaned first. Operator data under `~/ .tmx/` is preserved.

How to verify the uninstall worked

```
# rc snippet block gone:
grep -c '— vasic-digital optimized tmux —' ~/.bashrc ~/.zshrc
# expect: 0 in each

# init file gone:
ls -la scripts/tmx-shell-init.sh 2>&1
# expect: No such file or directory

# wrapper gone:
ls -la scripts/tmx 2>&1
# expect: No such file or directory
```

8. Last verified

2026-05-22 on:

- Darwin arm64, macOS 15.x, bash 3.2 / zsh 5.9, tmux 3.6a, Go 1.22;
- ALT Linux 11, kernel 6.12, systemd 258, bash 5.x, tmux 3.6a, Go 1.22.

Reproducible from a fresh clone via:

```
git clone --recurse-submodules git@github.com:vasic-digital/
tmux.git ~/Projects/tmux
cd ~/Projects/tmux
bash scripts/install_deps.sh    # Linux only
```

```
bash scripts/setup.sh
```

```
bash scripts/tests/run_all.sh # full suite — expect GREEN
```

All §11.4.50 deterministic-consistency loops (3 iterations identical evidence-hash) GREEN; all §11.4.81 cross-platform branches GREEN on both platforms; §1.1 paired-mutation harness reports caught>0 escaped=0.